

Constructed demonstration. All numerical records are teaching fixtures and must not be presented as real research evidence.

Early window records with revision-checked replacement

Abstract

A collector that emits once must either wait for late packets or leave its early output incomplete. We extend an existing collector with R5: it first emits after a configured 5 s wait, retains window state until 120 s after closure, and sends complete replacement records when accepted late packets change the aggregate. The receiver accepts only increasing revisions and updates count and sum together. In a constructed bursty replay of 48 windows, R5 improves from 31 initially exact windows to 48 finally exact windows through 17 corrections. The one-shot I5 remains at 31; the one-shot F120 is exact in all 48 after its 120 s wait. R5 retains 124 KiB, versus 96 KiB for F120 and 12 KiB for I5. When needed packets arrive beyond retention, both R5 and F120 end at 44 exact windows. These teaching fixtures illustrate earlier provisional output with bounded repair and additional state and transmissions; they are not real research evidence.

1 Introduction

A sample collector reports aggregates for event-time windows, but packets can arrive after their window closes. The earlier collector waits 120 s and then emits once. Reducing that wait to 5 s makes an output available sooner, while leaving later packets unreflected in it. Our change separates first emission from subsequent replacement: the collector retains each window's aggregate and sample identifiers, then emits a new version when an accepted late packet changes it. This also requires receiver-side handling because aggregate records can be duplicated or delivered out of order. We examine the resulting tradeoff using three constructed arrival profiles and one diagnostic. The work concerns record emission, replacement and expiry in this collector. It does not introduce event-time windows, an aggregation formula, watermarks, deduplication, transactions or version numbers, and no systematic literature comparison supports an external novelty claim.

2 Method: retained windows and complete replacements

The earlier implementation already partitions samples into 20 s windows by event timestamp and computes their count and sum. Each input packet has a stable (stream_id, sample_id). For each open or retained window, the prototype keeps its aggregate and a set of seen sample IDs: a newly seen ID updates count and sum, while a repeated input packet is ignored. Every emitted record contains (stream_id, window_end, revision, count, sum). The receiver derives the displayed mean from the record's count and sum using Eq. (1). Input-packet deduplication is separate from handling repeated deliveries of these emitted records.

$$\bar{x}_w = S_w / N_w \quad (1)$$

All variants use the same event-time closure rule. F120 waits 120 s after closure and emits once; I5 waits 5 s and emits once, without correcting that output. R5 first emits after 5 s and retains window state until 120 s after closure, regardless of subsequent arrival times. Each newly accepted late packet that changes the aggregate increments its revision and generates a complete replacement record. For the same stream and window, the receiver accepts a record only if its revision exceeds the stored revision, rejecting both repeated and older revisions. It replaces count and sum together in one transaction. At the fixed expiry, the collector evicts the aggregate and sample-ID set; later packets for that window are logged and rejected, never reassigned to the next window. The cutoff is unchanged across profiles.

Complete replacement prevents a duplicated correction from being added twice; the revision check prevents an older delivery from undoing a newer result. The transaction keeps count and sum from different revisions from being combined into a mean that no emitted record represents. An early development attempt considered additive correction deltas, but reasoning about duplicated deliveries exposed double counting; no measured result for that attempt is included. To isolate revision ordering, diagnostic U5 uses R5's generation side and complete-pair replacement, removing only the receiver's revision-order check. U5 was run only on the bursty trace. The prototype has one writer per stream. Reconnects, concurrent writers, persistence failures and long-running deployment were not tested, so these mechanisms do not establish complete exactly-once processing.

3 Replay and results

We constructed one replay for each variant/profile listed in Table 1. The on_time, bursty and beyond_retention profiles each contain 48 distinct windows, not 48 repeated benchmark trials; no timing confidence intervals or error bars are available. Exact agreement requires both count and sum to match the complete offline packet list, with final agreement checked after replay completion. First-output wait is a configured setting, not measured compute latency or execution speed. Peak retained state counts serialized window aggregates and sample IDs, not total process memory. Generated corrections count revisions after first emission, before any duplicate transmissions introduced by transport. Table 1 retains every supplied row, including the U5 diagnostic.

Table 1. Constructed replay records. Exact windows are initial/final out of 48; state is retained-state KiB.

Profile	Variant	Wait s	Exact initial/final	Corrections	State KiB
on_time	F120	120	48/48	0	96
on_time	I5	5	48/48	0	12
on_time	R5	5	48/48	0	124
bursty	F120	120	48/48	0	96
bursty	I5	5	31/31	0	12
bursty	R5	5	31/48	17	124
bursty	U5	5	31/38	17	124
beyond_retention	F120	120	44/44	0	96
beyond_retention	I5	5	36/36	0	12
beyond_retention	R5	5	36/44	8	124

For on_time, all three variants are exact in all 48 windows on first emission and need no corrections. In bursty, all needed packets arrive by 90 s after closure. F120 therefore first emits 48 exact windows at 120 s, whereas I5 and R5 each start with 31 at 5 s. R5's 17 generated corrections bring final agreement to 48; I5 remains at 31. The trace includes duplicated and out-of-order aggregate deliveries. With the revision-order check removed, U5 finishes at 38 exact windows because stale deliveries overwrite newer records in ten windows; its 17 generated corrections and 124 KiB state match R5's generation side. In beyond_retention, twelve windows are incomplete at 5 s. R5 repairs eight before its 120 s expiry, moving from 36 to 44 exact windows with eight corrections. F120 also finishes at 44, and I5 remains at 36. Four needed packets arrive at 180 s, beyond both 120 s cutoffs. R5 cannot repair those windows under the fixed rule. Across the supplied profiles, R5 retains 124 KiB, compared with F120's 96 KiB and I5's 12 KiB, and corrections add record transmissions. No alternative parameter setting or retuning was evaluated.

4 Conclusion

R5 changes when the collector can revise a reported window: a 5 s first emission can be replaced until state expires 120 s after closure. Retained sample IDs, complete-record replacement, increasing receiver revisions and atomic count/sum updates address different causes of incorrect output. In the constructed

bursty trace, this combination restores final agreement to F120's 48 windows while exposing the same initial incompleteness as I5. It costs more retained state and additional transmissions, and it shares F120's failure to include packets arriving beyond the cutoff. The U5 diagnostic supports the revision-order check under the supplied deliveries, not general system robustness. These constructed records establish neither production readiness nor general superiority over stream processing systems.

Sources and scope

The implementation notebook and results.csv supplied with this demonstration are the only sources. There is no literature review or external novelty claim. All supplied rows, including negative outcomes, remain part of the manuscript.